

Mira — Problem Statement & PRD

1. Problem Statement

Company: Nexora Pvt. Ltd., a London-headquartered software/IT services company (est. 2018, 120+ staff), has won the contract to run the **AI Adoption Project** for ABCDE Ltd., a mid-sized logistics and supply chain company — a 6-month engagement covering demand forecasting, route optimization, inventory management, and customer support.

Role: A newly-hired AI PM/TPM at Nexora owns this project end-to-end, working across data science, ML engineering, and product stakeholders to set scope, milestones, and delivery — with access to the project description, timeline, risk assessment, and task board.

The problem: Nexora's PMs/TPMs run multiple concurrent projects with a lean team, and the surrounding process work does not scale with them:

- Turning a high-level project description into a structured plan takes **3–4 hours of manual work per project**.
- Risk assessment is **inconsistent across projects** — some have detailed risk matrices, others have none at all.
- Compiling a weekly status report from task boards and meeting notes takes **1–2 hours**.
- Milestones get missed because there is **no proactive alerting** — issues surface only in retrospectives, after the cost is already sunk.
- Plans, status reports, and risk documents have **no standardized format** across the team, which makes them hard to compare, hand off, or audit.

The mandate: build **Mira**, an AI-powered project intelligence assistant that ingests raw project data and produces structured, actionable planning, risk, and status outputs a PM/TPM can use immediately — grounded in real project files rather than generic templates.

Validation against the actual build: Mira was implemented and evaluated against this exact brief. *Mira — Component Level Design Document* further details the requirements and core capabilities. The baseline test dataset (12 inputs, T1–T12) was run end-to-end before baselining the code base. Interim results are all captured via LangSmith tracing and a DeepEval suite.

2. Product Vision & Goals

Vision: Give every PM/TPM at Nexora an AI teammate that turns raw project files into trustworthy, ready-to-use plans, risk assessments, and status reports — in minutes, in one consistent format, without manual compilation.

Target users: Nexora's AI PMs/TPMs (initial rollout target: 10 PMs/TPMs), each juggling multiple concurrent client engagements with a lean team.

Goals, tied directly to the brief's stated pain points:

Mira — Problem Statement & PRD

Current state (from Problem Statement)	Mira's goal
3–4 hrs manual work to turn a description into a plan	Generate a grounded, phased plan
Inconsistent risk assessments — some projects have none	Every risk request returns a categorized, register-grounded matrix
1–2 hrs to compile a weekly status report	Generate a status report directly from the live task board on request, bucketed by category (done/in-progress/blocked/at-risk/upcoming)
Missed milestones, no proactive alerts	Daily automated milestone check that flags at-risk items before they're missed, not in a retrospective
No standardized output format	Schema designed per each capability, enforced by each agent and re-verified before anything reaches the user

Non-goal for this vision statement: Mira does not replace the PM/TPM's judgment — it produces a grounded first draft and flags what's missing; a human still reviews and, for register writes for any new identified risks - explicitly approved (see Human-in-the-Loop Scope).

3. In Scope

Core capabilities (required, all three built)

1. **Project Plan Generation** — ingest a high-level request, return a structured plan (phases, activities, milestones, deliverables, timeline estimates), grounded in the actual project description. Built as the **Planner Agent (N11)**.
2. **Risk Assessment** — analyze project details and organizational events, return a categorized risk matrix (description, impact, mitigation) grounded in the live risk register. Built as the **Risk Assessor Agent (N16)**.
3. **Weekly Status Report** — ingest live task board data, return tasks by status plus blockers, at-risk items, and upcoming milestones, with an overall health rating. Built as the **Status Reporter Agent (N25)**.

Note: A user-facing UI. Mira is accessed via the n8n chat widget or a webhook, with all output additionally mirrored to a Slack channel.

Extended capabilities chosen (brief required at least one; five were built)

Capability	Why chosen over the alternatives offered
Milestone Alert System	Directly closes the brief's "missed milestones due to lack of proactive alerts" pain point — the highest-leverage extension relative to effort, since it reuses the same retrieval-tool pattern as the other three agents. Built as the Milestone

Mira — Problem Statement & PRD

Capability	Why chosen over the alternatives offered
	Tracker Agent (M2) , on a daily schedule trigger, posting flagged at-risk milestones to Slack automatically (no request needed).
RAG / Knowledge Retrieval	Chosen as the grounding mechanism for <i>all</i> agents, not a standalone feature — a two-pipeline (ingestion + retrieval) Pinecone setup keeps every agent's output traceable to a real source document rather than model memory.
Human-in-the-Loop	Scoped narrowly and deliberately: only the one truly consequential <i>write</i> action — appending a new risk to the shared register — is gated behind a Slack Approve/Deny step (N21b). Read-only outputs (plans, existing-risk matrices, status reports) are not gated, since gating every response would defeat the goal of turning a 3–4 hour manual task into a same-session tool. As an iterative development, we can extend this feature as a future development to re-loop for identifying the risks and getting back for approval (say loop 3 times max.).
Fine-Tuning	Applied specifically to the Risk Assessor (N16), the agent with the most confirmed bug/fix history (vague-statement handling, top-N enforcement, new-vs-existing routing) — trained on 9 real corrected examples from the build's own testing, rather than synthetic data.
Stakeholder Update Generator	Originally descope in the initial build, then added once the core three agents were stable, closing the brief's stakeholder-communication option at low incremental cost by reusing the Status Reporter's retrieval tool. It as well independently checks task due-dates so the agent can never silently omit a missed deadline — a defense-in-depth pattern adopted directly from a real bug caught in this exact branch during testing.

Extended capabilities considered but not built: Resource Allocation Advisor (over/under-allocation analysis) was listed as an option in the brief but descope in favor of depth on Milestone Alerts + Human-in-the-Loop + Fine-Tuning, judged the higher-value combination given the brief's own emphasis on missed milestones and inconsistent risk rigor as the costliest current pain points. Competitive/Market Analysis (the fifth listed option) was not pursued — it addresses a different problem (market positioning) than the operational PM/TPM pain points this brief targets.

For the rejected alternatives and rationale behind the technical decisions underlying this scope (orchestration pattern, retrieval strategy, guardrail design, fine-tuning, HITL scope, and more), see the document “Mira — Component Level Design Document” Alternative Approaches Considered.

Mira — Problem Statement & PRD

Orchestration pattern

Router/Dispatcher, chosen over Sequential Pipeline and Hierarchical: user requests genuinely vary in kind (plan vs. risk vs. status vs. unclear), so a single Classifier Agent routes each request to the specialist best suited to it, rather than running every request through every agent (Sequential) or introducing an extra decomposition layer for what are, in practice, three independent, non-decomposable specialist jobs (Hierarchical). Extended to support **multi-intent** requests (a single message can route to more than one specialist in parallel). For full rationale and diagrams: see the document “*Mira — Component Level Design Document*”.

4. Out of Scope

- **Task creation or editing.** Mira reads the task board and risk register; it does not create, reassign, or close tasks (Jira/Trello write-back was listed as optional in the brief and not built).
- **Stakeholder Update Generator and Resource Allocation Advisor** (see Scope for rationale).
- **Competitive/Market Analysis.**
- **Multi-project support beyond ABCDE Ltd.** The current Pinecone namespace, Google Sheets documents, and prompts are scoped to one client engagement; multi-tenant support (separate namespaces/registers per client) is a rollout consideration, not built.
- **Autonomous milestone remediation.** The Milestone Tracker flags at-risk items and suggests an action; it does not reassign owners or change dates itself.

5. User Stories & Acceptance Criteria

Plan Generation

As a PM/TPM, I want to describe a project at a high level and get a structured phased plan, so I don't spend 3–4 hours building one by hand.

- **Given** a request naming a specific, documented project/component (e.g. "generate a plan for the AI Adoption Project"), **then** Mira returns phases with activities, milestones, deliverables, and timeline estimates, each traceable to the retrieved project description — never invented.
- **Given** a vague request with no identifiable scope (e.g. "we want to build a chatbot") or only a bare duration ("a 2-week project"), **then** Mira asks for the missing scope/goals/timeline/team size instead of fabricating a plan (`sufficient_detail: false`).
- **Given** a real, retrieved numeric KPI/target in the source project description, **then** that number appears explicitly in the plan's phases — never softened or dropped.

Risk Assessment

As a PM/TPM, I want a consistent, categorized risk matrix for any project or event I ask about, so risk rigor doesn't depend on who happens to be running the project.

Mira — Problem Statement & PRD

- **Given** a request for existing/broad risks ("top 3 risks", "assess the project"), **then** Mira returns only risks that are real rows in the live register, capped to any requested count exactly.
- **Given** a report of a new event (e.g. a named person leaving), **then** Mira checks the register for a literal existing match; if none exists, it proposes a new risk entry and **requires explicit Slack approval** before it is written to the register.
- **Given** a question about an already-tracked situation, **then** Mira never creates a duplicate risk, even if phrased as a report.
- **Given** a vague statement with no identifiable project ("new project starting soon"), **then** Mira flags insufficient detail rather than inventing project-specific risks.

Weekly Status Report

As a PM/TPM, I want to ask for a status report — for everything, a sprint, or a person — and get accurate, current numbers, so I stop manually compiling task boards.

- **Given** a general check ("give me a status update"), **then** Mira returns the full report across all categories, health-rated Green/Amber/Red.
- **Given** a sprint- or category-scoped request, **then** only the requested categories/sprint are populated, computed against the live task board's real status/sprint fields (not the LLM's own recall).
- **Given** no real ask at all ("things are going fine"), **then** Mira asks a clarifying question rather than fabricating a report.
- **Given** the task board has no data, **then** Mira states that plainly rather than reporting a false Green.

Extended: Milestone Alert System

As a PM/TPM, I want to be warned about at-risk milestones before they're missed, not after.

- **Every day**, Mira checks all milestones against the current date and flags any that are overdue, or due within 7 days with an incomplete dependency, posting the alert with a reason and a suggested action to the team Slack channel automatically.

Extended: Human-in-the-Loop

As a PM/TPM, I want to approve any new risk before it's permanently added to the shared risk register.

- **Given** Mira identifies a genuinely new risk, **then** it is never written to the register until a team member clicks Approve in Slack; Deny leaves the register unchanged.

Extended: Stakeholder Update Generator

As a PM/TPM, I want a ready-to-send stakeholder update email for a sprint, grounded in the real task board, so I don't have to draft status updates by hand for every audience.

Mira — Problem Statement & PRD

- **Given** a request naming a sprint or reporting period, **then** Mira returns a subject line, a progress summary, and next steps grounded in retrieved task/timeline data — never invented.
- **Given** any in-scope task has passed its due date and is not Done, **then** the email must explicitly name it and its missed deadline.

6. Functional Requirements

The system shall provide the following capabilities. Full production input/output schemas for each are documented in the document *“Mira — Component Level Design Document”*.

FR-1 — Project Plan Generation. Accept a plan request and return a structured, phased plan (activities, milestones, deliverables, timeline estimates) grounded in the project's own description; ask a clarifying question rather than fabricate a plan when the request lacks sufficient detail.

FR-2 — Risk Assessment. Accept a risk-related request and return a categorized risk matrix (risk, impact, mitigation) grounded in the live risk register; propose new risks for approval rather than write them automatically.

FR-3 — Weekly Status Reporting. Accept a status request and return current task status bucketed by category (done / in-progress / blocked / at-risk) plus an overall health rating, computed from the live task board.

FR-4 — Milestone Alerting. Run on a daily schedule and proactively flag milestones that are overdue or at risk, with a reason and a suggested action, without requiring a user request.

FR-5 — Human-in-the-Loop Approval. Require explicit human approval in Slack before any new risk is permanently written to the shared register; all other outputs remain read-only.

FR-6 — Stakeholder Update Generation. Accept a request scoped to a sprint or reporting period and return a subject line, progress summary, and next steps grounded in live task/timeline data, explicitly naming any task that is overdue.

FR-7 — Grounding & Traceability (cross-cutting). Every functional response must be grounded in retrieved project data; the system must never answer from assumptions about data it has not retrieved.

FR-8 — Consistent Output Format (cross-cutting). Each capability returns output in one fixed, predictable structure regardless of which run produced it, so downstream consumption is consistent.

7. Non-Functional Requirements

Requirement	How it's met
NFR-1 — Grounding / anti-hallucination	Every agent prompt enforces a strict no-hallucination principle; retrieval is scoped to each agent's own domain, and any output affecting numeric facts is independently re-verified against the live source data before being

Mira — Problem Statement & PRD

Requirement	How it's met
	returned. Full implementation detail: see the companion Component Level Design Document.
NFR-2 — Data freshness	The knowledge base is refreshed on a short, fixed schedule with an idempotent rebuild, so retrieval never drifts far from the source-of-truth project files.
NFR-3 — Auditability	Every response is posted to a single shared channel, creating one running record of every interaction regardless of entry point.
NFR-4 — Governed writes	The one mutating action (adding a new risk) requires human approval; every other output is read-only.
NFR-5 — Error resilience	Expected failures degrade gracefully with a clarifying response instead of a crash; unexpected failures are caught globally and alerted to the team separately from normal output.
NFR-6 — Consistency of format	Each capability returns one fixed output structure, so downstream consumption is deterministic regardless of which run produced it.
NFR-7 — Reliability under LLM unpredictability	Testing showed the model's own self-assessment of whether it had enough information, and its own categorization of task status, were not reliable enough to trust unsupervised — so those judgments are independently re-checked before reaching the user rather than trusted on a single pass.
NFR-8 — Cost/quota awareness	Observability tooling usage is monitored against its plan's tracing quota so it does not become a bottleneck as usage grows.

8. Success Metrics

Metric	Baseline (per Problem Statement)	Target with Mira
Time to produce a project plan from a description	3–4 hours manual	Reviewed and edited by the PM in minutes rather than built from scratch
Risk-matrix coverage across active projects	Inconsistent — some projects have no risk matrix at all	100% of projects with a Drive-ingested description/timeline get a grounded risk matrix on request
Time to produce a weekly status report	1–2 hours manual	Details deterministically re-verified against the live task board

Mira — Problem Statement & PRD

Metric	Baseline (per Problem Statement)	Target with Mira
Milestones missed with no advance warning	Discovered only in retrospectives	0 — every at-risk milestone (overdue, or due ≤ 27 days with an incomplete dependency) is flagged the morning via the daily Milestone Tracker
New-risk register accuracy	N/A (no governance today)	100% of new risk entries are human-approved in Slack before being written — zero un-reviewed writes
Baseline test coverage	N/A	14/14 baseline test inputs (T1–T14) run and evaluated against expected-output criteria, added specifically to exercise the Human-in-the-Loop Slack-approval flow

9. Evaluation Strategy

Judgement Limitations and workarounds built

LLM gatekeeper unreliability: the Risk and Status agents' own judgment of whether a request has enough detail/data, and their own status categorization, were not reliable enough to trust standalone — mitigated with independent, deterministic re-checks before output reaches the user, but this means every new specialist agent added in future will likely need the same defense-in-depth pattern, not prompt engineering alone.

Testing Strategies

Baseline test dataset: 14 functional test cases (T1–T14) covering a detailed plan, a vague plan, a detailed risk assessment, a vague risk assessment, a data-backed status report, a no-data status report, a top-N risk query, a blocked/at-risk task query, an edge-case plan request with minimal detail, a full status summary, an upcoming-milestones query, a stakeholder-update summary, and two Human-in-the-Loop new-risk cases requiring Slack approval — all run end-to-end through the live workflow. Exact dataset contents: see the document “Mira — Component Level Design Document”.

10. Assumptions

- Each onboarded PM/TPM's project already has a populated Google Drive folder, Risk Register sheet, and Task Board sheet in the expected format before the system is pointed at it.
- The system is scoped to a single client engagement (ABCDE Ltd.) — one Pinecone namespace and one pair of Google Sheets; multi-project/multi-tenant support is not assumed.
- Users have access to the shared Slack channel used for approvals and alerts.

Mira — Problem Statement & PRD

- Underlying LLM and vector-store API access (OpenAI, Pinecone) remains available and within budget, and observability tooling (LangSmith) stays within its current plan's tracing quota.
- A PM/TPM reviews agent output, and explicitly approves any new risk, before treating it as final — the system produces a grounded first draft, not an autonomous decision.